Database Design Document

Angel Blessy (2547213)
Vishwas Vashishtha (2547255)
Nirupama Vincent (2547236)

MCA 2025-27

CHRIST(Deemed to be University)
Bangalore Central Campus

Guide: Dr. NAIVED GEORGE EAPEN

TABLE OF CONTENTS

List of Contents

1. **Database Overview**
    a. Introduction to Supabase
    b. Reasons for Choosing Supabase
2. **Database Architecture**
    2.1 Client → Supabase API → PostgreSQL Database
    2.2 Use of REST and Realtime API
    2.3 Role-Based Access via Row Level Security (RLS)
3. **Tables and Schema Design**
    3.1 app_users
    3.2 profiles
    3.3 merchants
    3.4 advertisements (ads)
    3.5 services
    3.6 places
    3.7 reviews
    3.8 contact_messages
    3.9 audit_logs
4. **Table Structure**
    4.1 app_users
    4.2 profiles
    4.3 merchants
    4.4 ads
    4.5 services
    4.6 places
    4.7 reviews
    4.8 contact_messages
    4.9 audit_logs
5. **Relationships & Foreign Key Constraints**
    5.1 Overview
    5.2 Foreign Key Definitions
    5.3 Referential Integrity Enforcement
6. **Indexing Strategy**
    6.1 Primary Key Indexes
    6.2 Foreign Key Indexes
    6.3 Unique Constraint Indexes
    6.4 Query Filter Indexes
    6.5 Indexing Summary
7. **Row Level Security (Supabase Specific)**
    7.1 Student Policies
    7.2 Merchant Policies
    7.3 Admin Policies
    7.4 Superadmin Policies
    7.5 Audit Log Protection
8. **Data Validation Rules**
    a. Rating Constraints

    b. Ad Duration Constraints
    c. Status Constraints
    d. Role Constraints
    e. Email Uniqueness
    f. Nullable vs NOT NULL Rules
    g. Theme mode
9. **Data Flow Integration**
    a. Client Request
    b. JWT Validation
    c. RLS Evaluation
    d. Query Execution
    e. Response Handling
10. **Backup & Security Considerations**

- Automatic Backups
- Encrypted Connections
- No Password Storage
- Role-Based Access Control
- Service Role Key Protection
- SQL Injection Protection

Database Design Section – Recommended Content Structure

1. Database Overview

Supabase is an open-source Backend-as-a-Service platform built on top of PostgreSQL. It provides a fully managed relational database along with a suite of backend services including an auto-generated REST API, Realtime data streaming, authentication, and file storage — all accessible through a single unified platform without requiring us to manage any server infrastructure.

Unlike traditional backend setups where each service needs to be independently configured and maintained, Supabase bundles everything we need around a production-grade PostgreSQL instance, making it well-suited for applications that require structured data, role-based access, and real-time capabilities.

Relational Data Model. Our application manages multiple interconnected entities — users, merchants, ads, services, places, and reviews — with well-defined relationships between them. PostgreSQL, which powers Supabase, supports the foreign key constraints, joins, and relational queries our schema demands. A non-relational database would not handle these relationships as cleanly or efficiently.

Row Level Security for Role-Based Access. Our platform has four distinct roles — student, merchant, admin, and superadmin — each with different data access permissions. Supabase allows us to enforce these rules directly at the database level using Postgres Row Level Security policies, ensuring that access control is applied consistently regardless of how a request originates.

Clerk Integration. We use Clerk for user authentication. Clerk issues JWTs containing the user's role, which Supabase validates on every request. This allows our RLS policies to identify the caller and enforce the correct access rules without any additional middleware on our end.

Auto-Generated REST API. Supabase automatically exposes our database schema as REST endpoints via PostgREST. This eliminates the need for us to write and maintain standard CRUD routes, reducing our backend development overhead significantly.

Realtime Capabilities. Supabase's Realtime API allows us to push database change events to connected clients over WebSocket. We use this to notify merchants instantly when their ad is approved or rejected, improving the user experience without requiring polling or manual refresh.

Reduced Infrastructure Overhead. As a managed platform, Supabase handles database provisioning, scaling, backups, and security patching. This allows our team to focus on application development rather than infrastructure management, which is critical given our project timeline.

2. Database Architecture

2.1 Client → Supabase API → PostgreSQL Database

Our application follows a three-tier architecture where the client communicates exclusively through Supabase's managed API layer, which sits in front of our PostgreSQL database. No client ever interacts with the database directly. Every request passes through the API gateway, where the caller's identity is verified and access policies are evaluated before any query reaches PostgreSQL.

This design ensures that our security model is enforced at the infrastructure level rather than relying solely on application-side logic.

2.2 Use of REST and Realtime API

Supabase exposes our PostgreSQL schema through two API interfaces — REST and Realtime — both of which we use in our application.

The REST API is powered by PostgREST, which automatically generates endpoints from our live database schema. Standard HTTP verbs map directly to database operations, and every request carries the user's JWT issued by Clerk, which Supabase validates to identify the caller's role before executing the query.

The Realtime API uses Postgres logical replication to stream row-level change events to connected clients over WebSocket. We use this for the ad approval workflow — when an admin updates an ad's status, the merchant receives that change instantly on their end without any polling. This gives us live status feedback on a critical flow without additional infrastructure.

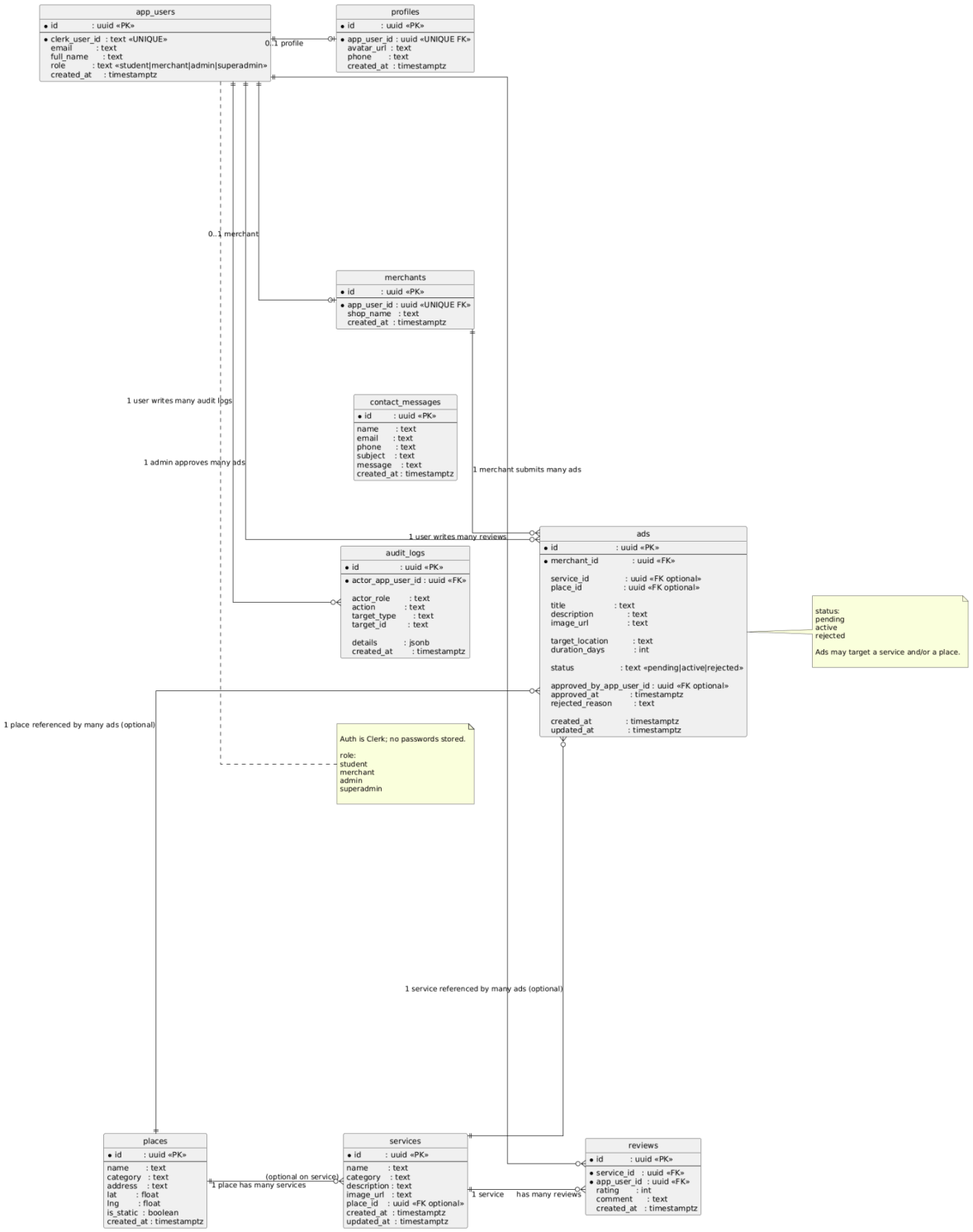
2.3 Role-Based Access via Row Level Security (RLS)

We enforce all data access rules using Postgres Row Level Security. RLS policies are defined per table and per operation, and are evaluated by PostgreSQL at query time. This means the API cannot return or modify rows that a policy prohibits, regardless of how the request was constructed — making it our most reliable enforcement layer.

Our application defines four roles — student, merchant, admin, and superadmin — each mapped to a distinct set of policies. Students can read public content and manage their own reviews. Merchants can create and manage their own ads and view their approval status. Admins can approve or reject ads and access platform-wide data. Superadmins have unrestricted access across all tables.

All policies reference the user's role from the Clerk-issued JWT, extracted via a shared helper function. This keeps policy definitions consistent across the schema and ties access control directly to the authenticated session.

3. Tables and Schema Design



3.1 app_users

Description The central identity table of our application. Every user on the platform — regardless of role — has a record here. Authentication is handled externally by Clerk, so no passwords are stored. The `clerk_user_id` column bridges the Clerk identity to our internal data model.

Attributes

Column	Type	Description
id	uuid	Primary key
clerk_user_id	text	Unique identifier from Clerk auth
email	text	User's email address
full_name	text	User's full name
role	text	One of: student, merchant, admin, superadmin
created_at	timestamptz	Record creation timestamp

Primary Key: `id`

Foreign Keys: None

Constraints

- `clerk_user_id` is UNIQUE
- `role` is constrained to student, merchant, admin, superadmin

Relationships

- Has one `profiles` record
- Has zero or one `merchants` record
- Writes many `audit_logs` records
- Writes many `reviews`
- Referenced by `ads` as the approving user

3.2 profiles

Description Stores extended user information that supplements the core identity in `app_users`. Each user has exactly one profile record. Holds avatar, phone, and other non-auth details.

Attributes

Column	Type	Description
id	uuid	Primary key
app_user_id	uuid	Reference to app_users
avatar_url	text	URL to user's profile image
phone	text	User's phone number
created_at	timestamptz	Record creation timestamp

Primary Key: `id`

Foreign Keys: `app_user_id` → `app_users.id`

Constraints

- `app_user_id` is UNIQUE, enforcing a strict one-to-one relationship with `app_users`

Relationships

- Belongs to one `app_users` record

3.3 Merchants

Description Represents a merchant entity on the platform. A merchant is a user with the merchant role who has registered a shop. This table holds merchant-specific information separate from the base user record.

Attributes

Column	Type	Description
id	uuid	Primary key
app_user_id	uuid	Reference to app_users
shop_name	text	Name of the merchant's shop

created_at	timestamptz	Record creation timestamp

Primary Key: `id`

Foreign Keys: `app_user_id` → `app_users.id`

Constraints

- `app_user_id` is UNIQUE, ensuring one merchant record per user

Relationships

- Belongs to one `app_users` record
- Submits many `ads`

3.4 Advertisements

Description Represents advertisements submitted by merchants. An ad goes through an approval lifecycle managed by admins — status moves from pending to active or rejected. An ad may optionally target a specific service, a specific place, or both.

Attributes

Column	Type	Description
id	uuid	Primary key
merchant_id	uuid	Reference to merchants
service_id	uuid	Reference to services (optional)
place_id	uuid	Reference to places (optional)
title	text	Ad title
description	text	Ad description
image_url	text	URL to ad image
target_location	text	Target location description
duration_days	int	Duration of the ad in days
status	text	One of: pending, active, rejected
approved_by_app_user_id	uuid	Admin who approved the ad (optional)

approved_at	timestamptz	Timestamp of approval
rejected_reason	text	Reason for rejection if applicable
created_at	timestamptz	Record creation timestamp
updated_at	timestamptz	Last updated timestamp

Primary Key: `id`

Foreign Keys

- `merchant_id` → `merchants.id`
- `service_id` → `services.id` (optional)
- `place_id` → `places.id` (optional)
- `approved_by_app_user_id` → `app_users.id` (optional)

Constraints

- `status` is constrained to pending, active, rejected

Relationships

- Belongs to one `merchants` record
- Optionally references one `services` record
- Optionally references one `places` record
- Approved or rejected by one `app_users` record

3.5 Services

Description Represents services available on the platform. A service belongs to a category and may be associated with a physical place. Services can be targeted by ads and reviewed by users.

Attributes

Column	Type	Description
id	uuid	Primary key
name	text	Name of the service
category	text	Category the service belongs to
description	text	Description of the service

image_url	text	URL to service image
place_id	uuid	Associated place (optional)
created_at	timestamptz	Record creation timestamp
updated_at	timestamptz	Last updated timestamp

Primary Key: `id`

Foreign Keys: `place_id` → `places.id` (optional)

Constraints: None beyond standard NOT NULL on required fields

Relationships

- Optionally belongs to one `places` record
- Has many `reviews`
- Referenced by many `ads`

3.6 Places

Description Represents physical or geographic locations on the platform. A place can have multiple services associated with it and can be targeted by ads. The `is_static` flag indicates whether the place is a fixed reference location.

Attributes

Column	Type	Description
id	uuid	Primary key
name	text	Name of the place
category	text	Category of the place
address	text	Physical address
lat	float	Latitude coordinate
lng	float	Longitude coordinate
is_static	boolean	Whether the place is a static reference
created_at	timestamptz	Record creation timestamp

Primary Key: `id`

Foreign Keys: None

Constraints: None beyond standard NOT NULL on required fields

Relationships

- Has many `services`
- Referenced by many `ads`

3.7 Reviews

Description Stores user-submitted ratings and comments for services. A review is written by a platform user and is associated with a specific service. Each user can review a given service once.

Attributes

Column	Type	Description
id	uuid	Primary key
service_id	uuid	Reference to services
app_user_id	uuid	Reference to app_users
rating	int	Numeric rating score
comment	text	Written review comment
created_at	timestamptz	Record creation timestamp

Primary Key: `id`

Foreign Keys

- `service_id` → `services.id`
- `app_user_id` → `app_users.id`

Constraints: None beyond standard NOT NULL on required fields

Relationships

- Belongs to one `services` record
- Written by one `app_users` record

3.8 contact_messages

Description Stores messages submitted through the platform's contact form. These are standalone records with no required association to a registered user, supporting both authenticated and anonymous submissions.

Attributes

Column	Type	Description
id	uuid	Primary key
name	text	Sender's name
email	text	Sender's email address
phone	text	Sender's phone number
subject	text	Message subject
message	text	Message body
created_at	timestamptz	Record creation timestamp

Primary Key: `id`

Foreign Keys: None

Constraints: None beyond standard NOT NULL on required fields

Relationships: Standalone — no foreign key relationships to other tables

3.9 audit_logs

Description An append-only event trail that records all significant actions performed on the platform. Every entry captures who did what, to which entity, and when. The `details` column stores a flexible JSONB payload for action-specific context such as before/after diffs or rejection reasons.

Attributes

Column	Type	Description
id	uuid	Primary key
actor_app_user_id	uuid	User who performed the action
actor_role	text	Role of the actor at time of action

action	text	Verb describing the operation
target_type	text	Entity type being acted upon
target_id	text	UUID of the target entity
details	jsonb	Structured payload with action context
created_at	timestamptz	Immutable event timestamp

Primary Key: `id`

Foreign Keys: `actor_app_user_id` → `app_users.id`

Constraints

- `created_at` is set by database default and never updated
- No DELETE or UPDATE permitted on this table

Relationships

- Belongs to one `app_users` record as the acting user

4. Table Structure

4.1 app_users

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- clerk_user_id (TEXT, UNIQUE, NOT NULL)
- email (TEXT, NOT NULL)
- full_name (TEXT, NOT NULL)
- role (TEXT, NOT NULL, CHECK role IN ('student', 'merchant', 'admin', 'superadmin'))
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

4.2 profiles

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- app_user_id (UUID, UNIQUE, NOT NULL, FK → app_users.id)
- avatar_url (TEXT)
- phone (TEXT)
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

4.3 merchants

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- app_user_id (UUID, UNIQUE, NOT NULL, FK → app_users.id)
- shop_name (TEXT, NOT NULL)
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

4.4 ads

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- merchant_id (UUID, NOT NULL, FK → merchants.id)
- service_id (UUID, FK → services.id)
- place_id (UUID, FK → places.id)
- title (TEXT, NOT NULL)
- description (TEXT, NOT NULL)
- image_url (TEXT)
- target_location (TEXT)
- duration_days (INT, NOT NULL)
- status (TEXT, NOT NULL, DEFAULT 'pending', CHECK status IN ('pending', 'active', 'rejected'))
- approved_by_app_user_id (UUID, FK → app_users.id)
- approved_at (TIMESTAMPTZ)
- rejected_reason (TEXT)
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

- updated_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

4.5 services

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- name (TEXT, NOT NULL)
- category (TEXT, NOT NULL)
- description (TEXT)
- image_url (TEXT)
- place_id (UUID, FK → places.id)
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())
- updated_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

4.6 places

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- name (TEXT, NOT NULL)
- category (TEXT, NOT NULL)
- address (TEXT, NOT NULL)
- lat (FLOAT, NOT NULL)
- lng (FLOAT, NOT NULL)
- is_static (BOOLEAN, NOT NULL, DEFAULT false)
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

4.7 reviews

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- service_id (UUID, NOT NULL, FK → services.id)
- app_user_id (UUID, NOT NULL, FK → app_users.id)
- rating (INT, NOT NULL, CHECK rating BETWEEN 1 AND 5)
- comment (TEXT)
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())
- UNIQUE (service_id, app_user_id)

4.8 contact_messages

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- name (TEXT, NOT NULL)
- email (TEXT, NOT NULL)
- phone (TEXT)
- subject (TEXT, NOT NULL)
- message (TEXT, NOT NULL)
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

4.9 audit_logs

- id (UUID, PRIMARY KEY, DEFAULT gen_random_uuid())
- actor_app_user_id (UUID, NOT NULL, FK → app_users.id)
- actor_role (TEXT, NOT NULL)
- action (TEXT, NOT NULL)
- target_type (TEXT, NOT NULL)
- target_id (TEXT, NOT NULL)
- details (JSONB)
- created_at (TIMESTAMPTZ, NOT NULL, DEFAULT now())

5. Relationships & Foreign Key Constraints

5.1 Overview

All foreign key relationships in our schema are enforced at the PostgreSQL level, ensuring referential integrity is maintained regardless of the application layer. Supabase does not allow orphaned records — if a referenced row does not exist, the operation is rejected by the database.

5.2 Foreign Key Definitions

- **profiles.app_user_id → app_users.id** Every profile record must belong to a valid user. This is a strict one-to-one relationship enforced by a UNIQUE constraint on `app_user_id`. If a user is deleted, their profile is deleted automatically via ON DELETE CASCADE.
- **merchants.app_user_id → app_users.id** A merchant record can only exist for a valid user. The UNIQUE constraint ensures one merchant account per user. ON DELETE CASCADE removes the merchant record if the associated user is deleted.
- **ads.merchant_id → merchants.id** Every ad must be submitted by a registered merchant. If a merchant record is deleted, all their ads are removed via ON DELETE CASCADE.
- **ads.service_id → services.id** An ad may optionally target a service. This is a nullable foreign key — the ad remains valid without it. ON DELETE SET NULL is applied so that if a service is deleted, the ad's service reference is cleared rather than the ad being removed.
- **ads.place_id → places.id** An ad may optionally target a place. Same behaviour as above — nullable, with ON DELETE SET NULL to preserve the ad record if the referenced place is deleted.
- **ads.approved_by_app_user_id → app_users.id** References the admin user who approved or rejected the ad. Nullable — only populated after a status transition. ON DELETE SET NULL is applied to preserve the ad record if the admin user is ever removed.
- **services.place_id → places.id** A service may optionally be associated with a physical place. Nullable foreign key with ON DELETE SET NULL, so removing a place does not cascade-delete its associated services.
- **reviews.service_id → services.id** Every review must reference a valid service. ON DELETE CASCADE is applied — if a service is deleted, all its reviews are removed with it.
- **reviews.app_user_id → app_users.id** Every review must be written by a valid user. ON DELETE CASCADE is applied — if a user is deleted, their reviews are removed as well.
- **audit_logs.actor_app_user_id → app_users.id** Every audit entry must reference the user who performed the action. ON DELETE RESTRICT is applied here

intentionally — we do not allow a user to be deleted if they have audit log entries, preserving the integrity of the event trail.

5.3 Referential Integrity Enforcement

PostgreSQL enforces all foreign key constraints at transaction time. Any INSERT or UPDATE that references a non-existent parent row is rejected with a foreign key violation error before it reaches the application. This means our data integrity does not depend on application-level validation alone — it is guaranteed at the database level.

The following delete behaviours are applied across our schema:

Relationship	On Delete Behaviour	Reason
profiles → app_users	CASCADE	Profile has no meaning without the user
merchants → app_users	CASCADE	Merchant identity tied to user
ads → merchants	CASCADE	Ads cannot exist without a merchant
ads → services	SET NULL	Ad remains valid without a service target
ads → places	SET NULL	Ad remains valid without a place target
ads → app_users (approver)	SET NULL	Approval record preserved, actor nullable
services → places	SET NULL	Service remains valid without a place
reviews → services	CASCADE	Reviews have no meaning without the service
reviews → app_users	CASCADE	Reviews tied to the authoring user
audit_logs → app_users	RESTRICT	Event trail must never lose actor reference

6. Indexing Strategy

Indexes allow PostgreSQL to locate rows without scanning the entire table. Without indexes, every filter query performs a full sequential scan — acceptable at small scale but increasingly costly as data grows. We define indexes on columns that appear frequently in WHERE clauses, JOIN conditions, and ORDER BY operations.

6.1 Primary Key Indexes

PostgreSQL automatically creates a B-tree index on every primary key column. All `id` columns across every table are therefore indexed by default with no manual definition required.

6.2 Foreign Key Indexes

PostgreSQL does not automatically index foreign key columns. We define explicit indexes on all FK columns that participate in frequent JOIN operations or filter queries.

ads.merchant_id — Every query that fetches a merchant's ads filters on this column. Without an index, PostgreSQL scans the entire ads table for each merchant lookup. Indexing this column directly improves performance on one of our most frequent data access patterns.

ads.service_id and ads.place_id — Used when filtering ads by their target service or place, which is common in listing and discovery queries.

reviews.service_id — Every request to fetch reviews for a service filters on this column. As review volume grows, an index here directly impacts response time on one of the most frequently read endpoints in our application.

reviews.app_user_id — Used when fetching all reviews written by a specific user, relevant for profile pages and user activity views.

audit_logs.actor_app_user_id — Audit log queries are typically filtered by actor when admins review activity for a specific user. Since audit_logs is an append-only table that grows continuously, this index is critical for keeping those queries performant over time.

6.3 Unique Constraint Indexes

PostgreSQL automatically creates an index for every UNIQUE constraint. The following columns therefore carry implicit indexes at no additional cost:

- `app_users.clerk_user_id`
- `profiles.app_user_id`
- `merchants.app_user_id`

- `reviews(service_id, app_user_id)` — composite unique constraint

6.4 Query Filter Indexes

Beyond foreign keys, we index columns that appear directly in WHERE clauses across common application queries.

app_users.email — User lookups by email occur during authentication flows and admin searches. Indexing this column ensures these lookups remain fast regardless of how large the user table grows.

app_users.role — Role is queried frequently, both for RLS policy evaluation and for admin dashboards filtering users by type. Since every authenticated request indirectly touches this column through policy checks, an index here has broad performance impact.

ads.status — The most common ad query in our application filters by status — fetching active ads for public listing or pending ads for the admin review queue. This index has a direct impact on homepage and dashboard load times.

services.category and places.category — Category-based filtering is a core part of our service and place discovery flow. Indexing these columns ensures filter queries remain fast as the catalogue grows.

6.5 Summary

Index	Table	Column(s)	Purpose
Auto (PK)	All tables	id	Row lookup by primary key
Auto (UNIQUE)	app_users, profiles, merchants, reviews	clerk_user_id, app_user_id, (service_id, app_user_id)	Uniqueness enforcement and lookup
Foreign Key Index	ads	merchant_id	Fetch ads by merchant
Foreign Key Index	ads	service_id	Filter ads by service target

Foreign Key Index	ads	place_id	Filter ads by place target
Filter Index	ads	status	Active and pending ad listing queries
Foreign Key Index	reviews	service_id	Fetch reviews for a service
Foreign Key Index	reviews	app_user_id	Fetch reviews by a user
Foreign Key Index	audit_logs	actor_app_user_id	Activity lookup by actor
Filter Index	app_users	email	User lookup by email
Filter Index	app_users	role	Role-based filtering and RLS
Filter Index	services	category	Category filter on services
Filter Index	places	category	Category filter on places

7. Row Level Security (Supabase Specific)

Row Level Security is a PostgreSQL feature that we enforce across all tables in our schema. Rather than relying on application-level checks to restrict data access, RLS policies are evaluated by the database itself on every query — meaning no request, regardless of how it is constructed, can return or modify rows it is not permitted to touch.

Supabase integrates RLS with the JWT issued by Clerk. On every request, Supabase extracts the user's ID and role from the token and makes them available to policy conditions. Our policies reference these values to determine what each caller is allowed to see or change.

7.1 Student Policies

Students represent the general user base of our platform. Their access is intentionally limited to public content and their own personal data.

Students can read all active ads, services, and places. They cannot read pending or rejected ads submitted by other users. They can insert and update their own reviews, identified by matching `app_user_id` to the authenticated user's ID. They cannot modify or delete reviews written by other users. They can read and update their own profile record but cannot access profiles belonging to other users.

7.2 Merchant Policies

Merchants have write access scoped strictly to their own data. A merchant cannot view, edit, or delete another merchant's records under any circumstance.

Merchants can insert new ads where `merchant_id` resolves to their own merchants record. They can update and delete only ads they own. They can read the full detail of their own ads including status, approval timestamp, and rejection reason — fields that are not exposed to students. They cannot access other merchants' ads or any admin-level data.

7.3 Admin Policies

Admins have elevated read access across the platform and targeted write access for moderation actions.

Admins can read all ads regardless of status, including pending submissions awaiting review. They can update the `status`, `approved_by_app_user_id`, `approved_at`, and `rejected_reason` fields on any ad. They can read all

user records, contact messages, and audit logs. They cannot modify user roles, delete platform data, or access superadmin-restricted operations.

7.4 Superadmin Policies

Superadmins have unrestricted access across all tables and all operations. RLS policies are effectively bypassed for this role, allowing full platform management including user role changes and bulk data operations.

7.5 Audit Log Protection

The audit_logs table has a dedicated insert-only policy. No role — including admin — can update or delete audit entries through the client-facing API. Write access is restricted to server-side operations using Supabase's service role key, ensuring the event trail remains tamper-proof.

8. Data Validation Rules

- Validation in our schema is enforced at two layers — the database level through constraints and the application level through form validation. Database-level constraints are the authoritative source of truth and cannot be bypassed.
- **Rating constraint** — The `rating` column in the reviews table accepts only integer values between 1 and 5 inclusive. Any insert or update outside this range is rejected by the database before it is written.
- **duration_days constraint** — The `duration_days` column in the ads table is constrained to three allowed values: 7, 15, and 30. These correspond to our standard ad duration tiers. Any value outside this set is rejected at the database level.
- **status constraint** — The `status` column in the ads table accepts only pending, active, or rejected. This prevents any arbitrary status string from being written, ensuring the ad lifecycle state machine remains consistent.
- **role constraint** — The `role` column in app_users accepts only student, merchant, admin, or superadmin. This is enforced as a CHECK constraint and is the foundation of our entire RLS policy system.
- **email uniqueness** — The `email` column in app_users carries a UNIQUE constraint. Duplicate email registrations are rejected at the database level, preventing account conflicts independent of Clerk's own deduplication.
- **Nullable vs NOT NULL** — Columns that are structurally required — such as `merchant_id` on ads, `service_id` on reviews, and `actor_app_user_id` on audit_logs — are defined as NOT NULL. Optional references such as `service_id` and `place_id` on ads are explicitly nullable, reflecting the legitimate business case where an ad has no specific service or place target.
- **theme_mode** — If a theme settings record is introduced, the `theme_mode` column would be constrained to light and dark as the only accepted values, consistent with our UI toggle behaviour.

9. Data Flow Integration

- The following describes how data moves through our system from a client action to a database write and back.
- **Step 1 — Client Request.** The frontend application makes a request to Supabase using the client SDK. The request carries the user's active JWT in the Authorization header, issued by Clerk at the time of login.
- **Step 2 — JWT Validation.** Supabase validates the JWT signature and extracts the user's ID and role from the token claims. If the token is invalid or expired, the request is rejected before reaching the database.
- **Step 3 — RLS Evaluation.** PostgreSQL evaluates the applicable RLS policies for the requesting user's role and the operation being performed. Rows that the policy does not permit are filtered out of SELECT results or blocked entirely for write operations.
- **Step 4 — Query Execution.** If the request passes RLS evaluation, PostgreSQL executes the query against the relevant tables. All constraints — NOT NULL, UNIQUE, CHECK — are enforced at this stage.
- **Step 5 — Response.** Supabase returns a structured JSON response to the client. Successful operations return the affected rows or a confirmation payload. Errors return a structured object containing the Postgres error code and message, which our frontend handles to display appropriate feedback to the user.

10. Backup & Security Considerations

- **Automatic Backups.** Supabase performs automated daily backups of our PostgreSQL database on all paid plans. Point-in-time recovery is available, allowing us to restore the database to any specific moment within the retention window. This protects us against accidental data loss, failed migrations, or destructive queries.
- **Encrypted Connections.** All communication between the client and Supabase is transmitted over HTTPS. Connections to the PostgreSQL instance itself are encrypted via TLS. No data is transmitted in plaintext at any layer of our stack.
- **No Password Storage.** Our schema stores no passwords. Authentication is fully delegated to Clerk, which handles credential storage, hashing, and session management externally. This eliminates an entire class of security risk from our database design.
- **Role-Based Access.** Access to data is governed by our four-role model enforced through RLS. The principle of least privilege is applied throughout — each role can access only the data it needs to perform its function. No role has broader access than its responsibilities require, with the exception of superadmin which exists for platform management purposes.
- **Service Role Key Protection.** Supabase provides a service role key that bypasses RLS entirely. We use this key exclusively in server-side Edge Functions for operations such as audit log writes. It is never exposed to the client and is stored as a server-side environment variable.
- **SQL Injection Protection.** All database interactions go through the Supabase client SDK, which uses parameterised queries internally. Raw SQL is never constructed from user input in our application, eliminating SQL injection as an attack vector at the query layer.